



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

INTRODUCTION TO NUMPY

Professor: *Gabriel E. Cabrera*
Teaching Assistant: *Alex Den Braber*



1 Arrays

Objects such as lists (`list`) are very convenient because of their flexibility (they are mutable); however, this kind of structure has a cost: it uses a lot of memory and its performance is poor. Financial and/or scientific applications require high-performance operations on special structures. One of the most important data structures is the *array*. *Arrays* usually organize other objects of the same data type both in rows and in columns.

If we assume that the most relevant data types are `float` and `int` (the idea can be extended to other data types), a one-dimensional **array** would represent, mathematically speaking, a row or column vector (depending on how it is defined). The most common cases are matrices, which would be a two-dimensional **array** $i \times j$ (row and column).

We therefore need a class capable of working efficiently with *arrays*. The solution is the **NumPy** library with its `ndarray` class. If we want to create the matrix X :

$$X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

With lists (`list`):

```
x = list([[1,2,3],[4,5,6],[7,8,9]])  
x
```

```
## [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Where each element nested in the list would be a row of the matrix X and each element of the nested element a column. If we want to see the value at position 2×2 :

```
x[1][1]
```

```
## 5
```

However, Python interprets the variable X as a list and not as an **array**. Using `numpy`

```
import numpy as np
```

```
mat_x = np.array(x) # the array is generated  
mat_x
```

```
## array([[1, 2, 3],  
##        [4, 5, 6],  
##        [7, 8, 9]])
```

Now the variable `mat_x` is an *array* and as such it has the following attributes:

- Dimensions:

```
mat_x.ndim # it has two dimensions
```

```
## 2
```

- The values of the dimensions:

```
m, n = mat_x.shape # number of rows and columns
m, n
```

```
## (3, 3)
```

- All the elements:

```
mat_x.size
```

```
## 9
```

- New class:

```
type(mat_x)
```

```
## <class 'numpy.ndarray'>
```

the `range()` version that is *built-in* in Python exists in **NumPy** as `arange()`.

```
np.arange(1,10).reshape(-1,3)
```

```
## array([[1, 2, 3],
##        [4, 5, 6],
##        [7, 8, 9]])
```

The library documentation can be found at <https://numpy.org/doc/stable/reference/>.

2 NumPy

1. Generate the following matrix:

$$A = \begin{pmatrix} 3 & 0 & 2 \\ 2 & 0 & 2 \\ 0 & 1 & 1 \end{pmatrix}$$

- a. Find the transpose of the matrix A .
- b. Find the inverse matrix of A .
- c. Show that the following holds:

$$AB = BA = I$$

Where B is the inverse matrix of A and I the identity matrix.

- d. Using matrices, compute the mean of A .
2. Generate the following matrices:

$$C = \begin{pmatrix} 2 & 4 \\ 5 & -6 \end{pmatrix} \quad \text{and} \quad D = \begin{pmatrix} 9 & -3 \\ 3 & 6 \end{pmatrix}$$

- a. Obtain the sum of C and D .
 - b. Obtain the dot product of C and D .
3. Write a function that multiplies (not the dot product) two matrices.

3 Random Numbers

NumPy has its own pseudo-random number generator (Mersenne Twister). The generator is a mathematical function that produces a sequence of “random numbers”. It takes a parameter to start the sequence of numbers, called the seed (**seed**). The function is deterministic, that is, with the same seed it will always produce the same sequence of numbers (the choice of the seed does not matter).

1. Generate 10000 values from a standard normal distribution $\sim N(\mu = 0, \sigma = 1)$.
 - a. From the values created, generate a matrix E whose dimensions are 100×100 . What is the difference between **reshape()** and **resize()**?
 - b. “Flatten” the matrix E and verify that the data are indeed distributed with $\mu = 0$ and $\sigma = 0$.

4 System of Equations

1. Use matrices to solve the following systems of equations:
 - a.

$$\begin{aligned}a + b + c &= 6 \\ 3a - 2b + c &= 2 \\ 2a + b - c &= 1\end{aligned}$$

b.

$$\begin{aligned}3a + 4b - 5c + d &= 10 \\ 2a + 2b + 2c - d &= 5 \\ a - b + 5c - 5d &= 7 \\ 5a + d &= 4\end{aligned}$$